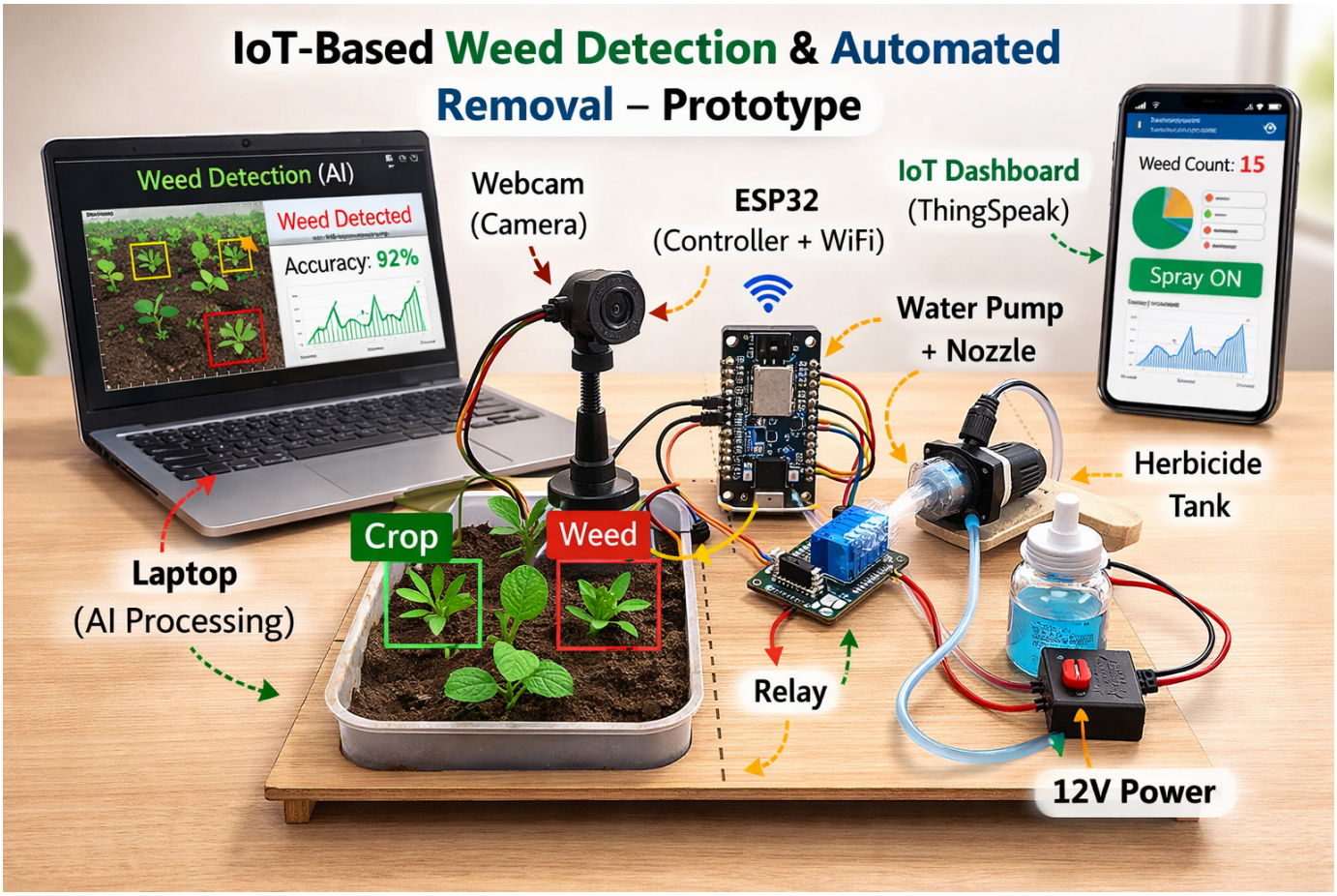
oT-Based Weed Detection & Automated Physical Cutting Syster

Major Project — Final Year Engineering (Low-Cost Prototype)

Prepared by: Team (Your Names Here)
Guide: (Your Guide Name) — Department of (Your Dept)

Date: March 2, 2026



Abstract

This document is a complete project report for an IoT-based weed detection and automated physical cutting system designed as a budget-friendly final-year major project. The prototype uses a webcam for image capture, laptop-based AI for detection, an ESP32 for IoT communication and actuator control, and a low-cost cutting mechanism for physical removal of weeds. The report includes system architecture, component list, bill of materials with estimated costs, software design, AI model workflow, wiring and control logic, safety considerations, testing plan, timeline and viva preparation material.

Introduction

Weeds reduce crop yield by competing for nutrients, water and light. Manual removal is labour-intensive. This project provides an affordable prototype that detects weeds using computer vision and removes them physically using a controlled cutting mechanism. The design prioritizes safety, low cost, and demonstrable functionality suitable for academic evaluation and future scaling. The system is intended for small-scale demo beds; in future it can be integrated into autonomous platforms.

Objectives

Primary objectives are to: detect weeds in real-time using an image-based AI model; trigger a physical cutting actuator to remove detected weeds; log events and sensor data to an IoT dashboard; provide manual override and safety features; and document performance (accuracy, spray/ cut frequency) for evaluation.

- Real-time weed vs crop detection (webcam + laptop AI).
- Automated physical cutting of detected weeds via ESP32-controlled motor.
- IoT logging (ThingSpeak/Blynk) for detection counts, soil moisture and system status.
- Safety mechanisms: emergency stop, enclosure and timed activation.
- Create full report, circuit diagrams, code and demo for viva.

Literature Survey (brief)

Relevant literature includes machine-vision-based weed detection research (YOLO, Faster-RCNN), edge computing for agriculture, and automated weeding systems (mechanical & precision spraying). Common datasets include DeepWeeds and PlantVillage; many studies report high detection accuracies using YOLO-family models.

System Overview

System components are split into: Vision & AI (webcam + laptop running detection model), IoT & Control (ESP32, relay/MOSFET, motor driver), Actuation (12V DC motor with cutter), Sensors (soil moisture, ultrasonic), Power (12V battery) and Cloud (ThingSpeak/Blynk). The system flow: image capture → AI detection → command to ESP32 → motor activation → cut → log event.

Hardware Components

Hardware components used in the prototype (minimum required): - Webcam (USB) for image capture - Laptop for running the AI model (OpenCV + YOLO/TensorFlow) - ESP32 development board for WiFi and control - Relay module or motor driver for switching the cutting motor - 12V DC high-speed motor (with small metal cutting blade) - Mini motor mounting, protective enclosure and bracket - Soil moisture sensor, ultrasonic sensor (optional) - 12V battery or adapter, wiring, connectors, switches, emergency stop

- Use a small circular blade (diameter 25–40 mm) fixed to motor shaft with secure nut.
- Prefer a low to moderate RPM motor; control activation time to 1–3 seconds per cut.
- Include an emergency stop (hard-switch) and transparent shield around blade.

Bill of Materials & Estimated Budget

Estimated Bill of Materials (approximate prices in INR): 1. Webcam (USB) — 600 2. ESP32 Dev Board — 500 3. Relay module / Motor driver — 150 4. 12V DC cutting motor — 400 5. Metal cutting blade (small) — 250 6. Mini motor mounting + bracket — 200 7. Soil moisture sensor — 50 8. Ultrasonic sensor — 150 9. 12V battery / adapter — 700 10. Wiring, switches, emergency stop, enclosure materials — 800 11. Wooden base / chassis — 400 Subtotal (approx): ■4,200 Contingency & misc: ■1,800 Total estimated cost: ■6,000 – ■9,000 (depending on component choices and sourcing).

Software Components

Software stack and tools: - AI: Python, OpenCV, PyTorch or TensorFlow, YOLOv5/YOLOv8 recommended for fast detection. - Microcontroller: Arduino framework for ESP32 (PlatformIO or Arduino IDE). - IoT: MQTT or HTTP to upload detection events to ThingSpeak/Blynk. - Languages: Python (detection & server), C/C++ (ESP32 firmware). - Libraries: OpenCV, numpy, paho-mqtt, requests, TensorFlow/PyTorch.

AI Model & Training Workflow

AI model workflow: 1. Dataset: collect images of crop and weed in similar lighting (or use public datasets for transfer learning). 2. Label images (crop vs weed) using LabelImg or Roboflow. 3. Train a YOLOv5 model on the labeled dataset; use augmentation to improve robustness (brightness, flip, rotate). 4. Export model and run inference on laptop with OpenCV. 5. On detection of 'weed' with confidence > threshold (e.g., 0.6), send command to ESP32 via HTTP/MQTT to activate cutter.

Control Logic

Control logic (high level): 1. Continuously capture frames from webcam. 2. Run object detection; for each detected bounding box labeled 'weed' check centroid location. 3. If weed is within reachable demo area and not recently activated (debounce), publish command to ESP32: {'action':'cut','x':X,'y':Y}. 4. ESP32 receives command, engages motor driver/relay for a short duration (1–3 sec) to spin blade. 5. Log event, increment weed count and update cloud dashboard.

Wiring & Interfacing

Wiring & interfacing (simplified): - ESP32 Vcc ← 5V regulated supply (or USB) - ESP32 GND ← common ground with motor power supply - Relay IN ← ESP32 digital output (GPIO) - Relay COM/NO ← 12V motor power line (motor positive through relay) - Motor negative ← motor power negative (shared ground) - Soil moisture sensor ← ESP32 analog input - Emergency stop placed on motor positive line to cut power instantly Important: use flyback diodes or snubber circuits with inductive loads; follow motor driver recommendations. For higher safety use a MOSFET-based driver with proper heat dissipation.

Code Snippets (abbreviated)

Key code snippets (abbreviated) — include in annex for full code. 1) Python (detection & command): ```python import cv2 from yolov5 import YOLOv5 # pseudocode model = YOLOv5('best.pt') cap = cv2.VideoCapture(0) while True: ret, frame = cap.read() preds = model.predict(frame) for p in preds: if p['label']=='weed' and p['confidence']>0.6: send_command_to_esp32() # display frame with boxes ``` 2) ESP32 (Arduino sketch) — pseudocode: ```c void loop(){ if(mqttMessageReceived=="cut"){ digitalWrite(RELAY_PIN, HIGH); delay(1500); // 1.5 sec digitalWrite(RELAY_PIN, LOW); } } ```

Safety Considerations

Safety and ethical considerations: - Always include a protective transparent shield covering the cutting blade. - Use low-power blades designed for delicate plant cutting in demos. - Add mechanical interlock and emergency stop to cut power immediately. - During testing, use water or soft demo plants; avoid live harmful chemicals. - Ensure wiring insulation and fuse protection for battery lines.

Testing & Validation

Testing & Validation plan: - Unit tests: camera frame rate, ESP32 connectivity, relay switching. - Integration tests: detection-trigger-actuation cycle in controlled demo tray. - Accuracy validation: test on N=200 sample images; compute precision, recall and F1 score. - Robustness: test under different lighting and background conditions. - Record logs of cut events; compute false-positive and false-negative rates.

Project Timeline

Suggested 8-week timeline (team-of-4): Week 1: Literature survey, finalise design & components Week 2: Dataset collection & labeling Week 3: Model training & initial inference setup Week 4: Hardware procurement & basic integration Week 5: Implement ESP32 control & IoT logging Week 6: Integrate cutting mechanism & safety features Week 7: Testing, data collection & performance evaluation Week 8: Report writing, presentation & viva preparation

Expected Results

Expected results: - Detection accuracy (on demo dataset): 85–95% depending on data quality. - False positives kept low by confidence thresholding and debounce logic. - Cutting actuation success rate: >90% when weed within reachable region. - Demonstrable IoT logs of events and sensor data.

Viva Preparation & Sample Q&A

Viva preparation (sample Q&A): Q: Why cutting over spraying? A: Cutting avoids chemical use and shows mechanical actuation skills; it's more demonstrative in a major project. Q: How do you ensure you don't cut crop plants? A: Use accurate detection model, limit action region, and use manual override/emergency stop. Future versions can integrate depth sensing. Q: Power considerations? A: Use 12V battery with fuse; measure current draw; include regulator for ESP32 and USB for laptop.

Conclusion

This report presents a feasible, low-cost major project that combines AI, IoT, and electromechanical actuation to automatically detect and physically remove weeds. The design balances cost, safety and demonstrability, and can be extended to autonomous platforms or scaled up with more advanced actuators and edge AI hardware.

References

References & resources (examples): 1. Glenn, et al., 'DeepWeeds: A Multiclass Weed Species Dataset for Deep Learning', 2019. 2. Jocher, Ultralytics YOLOv5 docs and tutorials. 3. PlantVillage dataset. 4. ESP32 documentation and Arduino core resources. 5. ThingSpeak and Blynk documentation for IoT dashboards.

Annex & Deliverables

Annex A: Full code files, dataset links and wiring diagrams should be included with the submission. Due to file size, include code as separate ZIP during submission; below are links you should prepare: - detection.py (python script for webcam and inference) - esp32_firmware.ino (Arduino sketch for MQTT/HTTP control) - BOM_receipts.pdf (optional proof of purchase) Annex B: Tips for demo setup: - Keep webcam fixed and focused on demo bed tray. - Use consistent lighting (LED lamp) to reduce false detections. - Calibrate the physical cutter position relative to image coordinates.